UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Business Galore: A business software

Julian David Celis Giraldo[1], Cataliza Ariza Ardila[2]
[1,2] University Francisco José de Caldas, Bogotá D.C, Colombia
20222020041[1], 20222020084[2]

Abstract

A common problem between clients and businesses that offer services is addressed by proposing a software-based solution. This involves analyzing the business model, understanding the relationship between the client and the store, and examining the business logic and requirements. An analysis is conducted using the object-oriented paradigm, developing diagrams to illustrate the findings.

Introduction

Are you tired of having to go to a beauty salon to ask if there's availability for a service and often getting a 'No!' in response, wasting your time? That's why we decided to develop software that allows users to schedule services at their preferred store, saving unnecessary trips and wait times. This is how Business Galore was born!

Now think that not only beauty salons require their clients to schedule appointments, but in general, any business that offers a service needs its users to book in advance in order to prepare their employees and assign them specific tasks for a specific client and time. This is how Business Galore was born. Initially, we aimed to connect clients with the schedule of the beauty salon, but the solutions developed in the software are flexible, allowing us to expand the market niche to other service-oriented businesses.

Goal

Develop an application that allows clients to connect with stores related to providing services, implementing design patterns and SOLID principles, under the object-oriented paradigm and to provide end users with a well-documented program that follows best coding practices and meets their requirements.

Develop

Starting from the identification of an existing need and the lack of an efficient solution to that need, we begin with the analysis of the general problem, which is to allow a user to schedule appointments for a service from a store. Thus, we conduct an analysis of the business model, identifying the existing need and the solutions already available in the market, and from there, we seek to make a difference. We identify the stakeholders, specifically the parties that interact directly when requesting a service: the client, the employee, and the administrator.

From a similar existing application in the market, we analyze user ratings, evaluating the shortcomings and advantages of those applications. This leads us to develop user stories, examining each user's role, the features they want to be launched, and the reasoning that supports those needs.

From this analysis, we employ CRC cards, where we assign an entity to each card. This entity indicates its responsibilities and its relationships or collaborations with other entities in the system. This will allow us to better understand what classes we require, as well as limit the scope of the app. Once developed, the CRC cards will help us identify the classes we will use in our project, their attributes, and the responsibilities will give us a starting point for the methods they will require. Additionally, they will help us determine the relationships between concrete and abstract classes.

The proposed solution in the class diagram is:

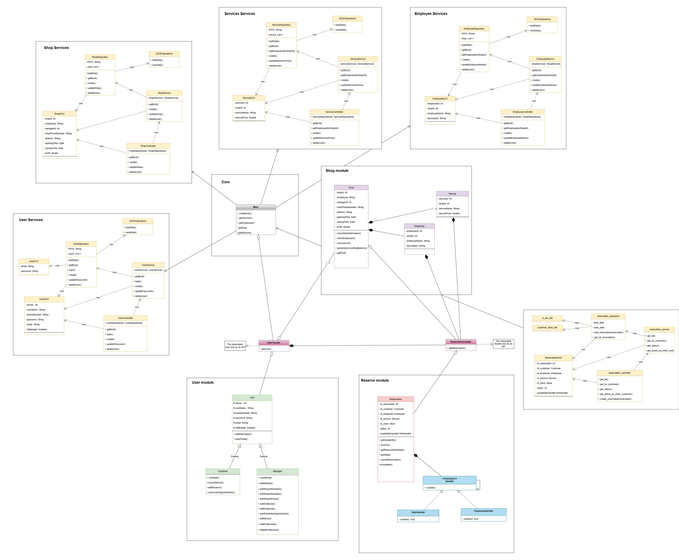

Figure 1: Diagram Class

Conclusions

Throughout the development of this document, the importance of design patterns, software architecture, and best programming practices has been evident. When properly implemented, these elements contribute to a more maintainable, robust, flexible, and clean codebase, among other benefits.

The use of UML diagrams, user stories with acceptance criteria, and CRC cards has streamlined code construction, making the development process faster and more structured. These tools have helped ensure that the final product aligns closely with the needs and expectations of the end user. By leveraging APIs, we were able to develop our project across different programming environments, allowing us to work more efficiently while taking advantage of the best features each environment has to offer.

One of the most critical aspects of application development is organization, especially in team-based projects. Implementing an agile methodology significantly improved our workflow and collaboration throughout the project. Applying SOLID principles in object-oriented projects is fundamental to achieving a well-structured and scalable program.

Additionally, the use of Docker simplified the creation, deployment, and management of our application through containerization. This was particularly useful in our case, where multiple backends needed to interact seamlessly. Docker's scalability also makes it an ideal choice for microservices-based applications like ours.

Acknowledgements

This work was primarily carried out by the students with their own resources.